

Iniciado em	segunda-feira, 13 jan. 2025, 14:13
Estado	Finalizada
Concluída em	segunda-feira, 13 jan. 2025, 15:39
Tempo empregado	1 hora 26 minutos
Avaliar	5,5 de um máximo de 10,0(55%)

Questão 1

Incorreto

Atingiu 0,0 de 1,0

O código a seguir foi compilado com OpenMP e o binário tem o nome t1. Com base no código fonte, analise as afirmações e marque V para as verdadeiras e F para as falsas.

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(void) {
5      printf(" %d\n", omp_get_num_threads());
6      #pragma omp parallel
7      {
8
9      }
10     printf("%d\n", omp_get_max_threads());
11     return 0;
12 }
```

I - A execução com o comando **OMP_NUM_THREADS=4 t1** vai imprimir o valor 4 na linha 5 e o valor 12 na linha 10, se o computador onde esse programa estiver rodando tiver 12 núcleos

II - Este programa vai imprimir sempre o valor 1 na linha 5, independente do número de threads definidas na variável **OMP_NUM_THREADS**

III - O comando da linha 10 vai imprimir sempre o valor 1, uma vez que este está fora da região paralela definida pelo **pragma omp parallel**

- ☐ a. Apenas as afirmativas II e III estão corretas
- ☒ b. Apenas as afirmativas I e II estão corretas ✖
- ☐ c. Apenas as afirmativas I e III estão corretas
- ☐ d. Apenas a afirmativa I está correta
- ☐ e. Nenhuma das opções consegue julgar as afirmativas apresentadas

Sua resposta está incorreta.

A resposta correta é:

Nenhuma das opções consegue julgar as afirmativas apresentadas



Questão 2

Completo

Atingiu 1,0 de 2,5

Paralelização com processos e threads

Dada uma lista de números de entrada, produza a soma desses elementos por meio de um programa MPI/OMP, considerando o seguinte:

- A lista de números deve ser dividida uniformemente entre os processos MPI gerados em tempo de execução, incluindo o MASTER. Por exemplo, se a quantidade de números da lista é igual a 36 e o programa for instanciado com 3 processos, cada um deles deverá tratar um subgrupo de 12 números.
- Cada processo deve criar tantas threads quantas forem identificadas no arquivo de entrada. Por exemplo, se um processo tem um subgrupo com 12 números e a quantidade threads/processo for 2, cada thread do referido processo deve cuidar de um subgrupo menor de 6 números.

Entrada

Na primeira linha do arquivo de entrada está a quantidade de threads que devem ser instanciadas em cada processo MPI que for ativado. Na segunda linha do arquivo de entrada está a quantidade de números a serem trabalhados. Na terceira linha do arquivo de entrada está a lista de números a serem somados.

Saída

O arquivo de saída contém o valor relativo à soma da lista de números do arquivo de entrada

Restrições

- Na parte MPI do programa, usar apenas as primitivas MPI_Send, MPI_Recv
- Na parte do OMP não é permitido usar o pragma de paralelismo com a opção reduction para consolidar resultados
- Para uma determinada entrada, o programa deve produzir sempre a mesma saída, independente da quantidade de processos MPI instanciadas em tempo de execução

Exemplo de Entrada 1

```
3
12
17 15 26 83 97 44 66 91 32 5 22 78
```

Exemplo de Saída 1

```
576
```

Exemplo de Entrada 2

```
4
8
126 18 77 82 6 21 11 20
```

Exemplo de Saída 2

```
361
```

Exemplo de Entrada 3

```
1
6
177 812 25 14 98 19
```

Exemplo de Saída 3

1145

Obs.: o arquivo de código deve ser nomeado com a matrícula do aluno, por exemplo, 2019000894.c

 [190089601.c](#)

Comentário:

Questão 3

Correto

Atingiu 1,0 de 1,0

O código a seguir foi compilado com OpenMP e o binário tem o nome t1. Com base no código fonte, analise as afirmações e marque V para as verdadeiras e F para as falsas.

```
1  include <stdio.h>
2  #include <omp.h>
3  #include <string.h>
4  #define MAX 100
5
6  int main(int argc, char *argv[]) {
7
8      #pragma omp parallel
9      {
10         int soma=0;
11         #pragma omp for
12         for (int i=0;i<MAX;i++) {
13             soma +=omp_get_num_threads()*i;
14         } /* fim-for */
15         printf("Thread[%d] iterou %d vezes\n",
16                omp_get_thread_num(), soma);
17     } /* fim omp parallel */
18     return 0;
19 }
```

- I - A execução com o comando **OMP_NUM_THREADS=4 t1** vai imprimir que cada thread foi executada 25 vezes
- II - Se este programa for acionado tendo a variável **OMP_NUM_THREADS** um valor maior do que o número de núcleos da máquina, apenas as threads equivalentes ao número de núcleos serão criadas
- III - Se o programa for executado numa máquina com 10 núcleos de processamento e a variável **OMP_NUM_THREADS** estiver com valor igual a 20, o programa não será ativado

- ☐ a. Apenas a afirmativa II está correta
- ☒ b. Todas as afirmativas estão erradas ✓
- ☐ c. Apenas as afirmativas II e III estão corretas
- ☐ d. Apenas a afirmativa III está correta
- ☐ e. Apenas a afirmativa I está correta

Sua resposta está correta.

A resposta correta é:

Todas as afirmativas estão erradas

Questão 4

Completo

Atingiu 2,5 de 2,5

O objetivo do código a seguir é fazer a impressão do texto repetidas vezes, em função da constante MAX e da variável number, que é calculada a cada iteração do while.

```
1  √ #include <sys/time.h>
2  #include <unistd.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #define MAX 1000
6  √ int main (void) {
7      char texto_base[] = "abcdefghijklmnopqrstuvwxyz 1234567890 ABCDEFGHIJKLMNOPQRSTUVWXYZ";
8      /* a variavel indice aponta para o primeiro caracter do texto */
9      int *indice=(int *) malloc (sizeof(int));
10     *indice = 0;
11     struct timeval tv;
12     int number, tmp_index, i, cont=0;
13     √ while(cont<MAX) {
14         gettimeofday (&tv, NULL );
15         number = ((tv.tv_usec / 47) % 3) + 1;
16         tmp_index = *indice;
17         for (i = 0; i < number; i++)
18             if( ! (tmp_index + i > sizeof(texto_base)) )
19                 fprintf(stderr, "%c", texto_base[tmp_index + i]);
20         *indice = tmp_index + i;
21     √ if (tmp_index + i > sizeof(texto_base)) {
22         fprintf(stderr, "\n");
23         *indice = 0;
24     } /* fim-if */
25     cont++;
26 } /* fim-while */
27 printf("\n");
28 return 0;
29 } /* fim-main */
```

Crie uma versão OpenMP deste código com apenas três threads, de modo que a lógica original se mantenha, mas a impressão do texto ocorra de modo colaborativo (com divisão de trabalho mais igual possível) entre as threads.

Obs.: O arquivo de resposta deve ter como nome a matrícula do aluno, por exemplo: 202000894.c

 [190089601.c](#)

#include <sys/time.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <omp.h>

#define MAX 1000

int *indice;

char texto_base[] = "abcdefghijklmnopqrstuvwxyz 1234567890 ABCDEFGHIJKLMNOPQRSTUVWXYZ";

void imprimeTexto(void) {

int tmp_index, i;

struct timeval tv;

int number;

gettimeofday(&tv, **NULL**);

number = ((tv.tv_usec / 47) % 3) + 1;

```

tmp_index = *indice;
for(i = 0; i < number; i++)
    if( ! (tmp_index + i > sizeof(texto_base)) ) {
        fprintf(stderr, "%c", texto_base[tmp_index + i]);
        usleep(1);
    } /* fim-if */

*indice = tmp_index + i;
if( tmp_index + i > sizeof(texto_base) ) {
    fprintf(stderr, "\n");
    *indice = 0;
} /* fim-if */
} /* fim-imprimeTexto */

int main() {
    indice = (int *) malloc(sizeof(int));
    *indice = 0;
    int cont=0;

    #pragma omp parallel num_threads(3)
    {
        printf("Thread %d iniciada...\n", omp_get_thread_num());
        sleep(1);

        /* Entrando no loop principal */
        while(cont<MAX) {
            #pragma omp critical
            imprimeTexto();

            #pragma omp atomic
            cont++;
        } /* fim-while */
    } /* fim-pragma */

    printf("\n");
    return 0;
} /* fim-main */

```

Comentário:

Questão 5

Incorreto

Atingiu 0,0 de 1,0

Analise o código a seguir.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <cuda_runtime.h>
4  #include <cuda.h>
5
6  __global__ void vecAdd(int *c) {
7      int i=threadIdx.x;
8      int j=blockIdx.x;
9      int k=blockDim.x;
10
11     c[i] = i+j+k;
12 } /* fim vecAdd */
13
14 int main(int argc, char *argv[]) {
15     int *c, *dc;
16     int n=atoi(argv[1]);
17     int size = n * sizeof(int);
18
19     cudaDeviceReset();
20     c = (int*) malloc(size);
21     cudaMalloc((void **) &dc, size);
22     vecAdd <<<2,n/2>>> (dc);
23     cudaDeviceSynchronize();
24     cudaMemcpy(c, dc, size, cudaMemcpyDeviceToHost);
25
26     printf("Resultado:\n");
27     for (int i=0; i<n; i++)
28         printf(" %d ", c[i]);
29
30     printf("\n");
31     cudaFree(dc);
32
33     return 0;
34 } /* fim-main */
```

Suponha que n (linha 16) é igual a 10, analise as afirmativas a seguir e marque a alternativa INCORRETA.

- ☐ a. Se for utilizada a fórmula $c[i+(j*k)]=i+j+k$, o vetor a ser impresso é 0 0 0 0 0 5 6 7 8 9
- ☒ b. Da forma como está, o vetor a ser impresso é 6 7 8 9 10 0 0 0 0 0 ✖
- ☐ c. Se for utilizada a fórmula $c[i*(j+k)]=i+j+k$, o vetor a ser impresso não é 5 6 7 8 9 0 0 0 0 0
- ☐ d. Se houver substituição do comando da linha 11 por $c[k] = i+j+k$, o vetor a ser impresso é 0 0 0 0 0 6 0 0 0 0
- ☐ e. Se houver substituição do comando da linha 11 por $c[j] = i+j+k$, o vetor a ser impresso é 5 6 0 0 0 0 0 0 0 0

Sua resposta está incorreta.

A resposta correta é:

Se for utilizada a fórmula $c[i+(j*k)]=i+j+k$, o vetor a ser impresso é 0 0 0 0 0 5 6 7 8 9

Questão 6

Incorreto

Atingiu 0,0 de 1,0

O código a seguir foi compilado com OpenMP e o binário tem o nome t1. Com base no código fonte, analise as afirmações e marque V para as verdadeiras e F para as falsas.

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main(int argc, char *argv[]) {
5
6      int i=0;
7      #pragma omp parallel
8      {
9          if (omp_get_thread_num() == 1)
10             i=i+10;
11     }
12     printf("i=%d\n", i);
13     return 0;
14 }
```

I - A execução com o comando **OMP_NUM_THREADS=1 t1** vai imprimir o valor zero para a variável i

II - Se na linha 7 for acrescentada a declaração **private(i)**, o binário equivalente acionado com o comando **OMP_NUM_THREADS= 2 t1** vai imprimir o valor 20

III - Ao suprimir a linha 9, o binário equivalente vai imprimir valores aleatórios para a variável i, desde que o número de threads seja maior que 1

- ☐ a. Nenhuma das opções apresentadas consegue julgar corretamente as afirmativas
- ☐ b. Apenas as afirmativas II e III estão corretas
- ☒ c. Apenas a afirmativa I está correta ✖
- ☐ d. Apenas as afirmativas I e II estão corretas
- ☐ e. Apenas as afirmativas I e III estão corretas

Sua resposta está incorreta.

A resposta correta é:

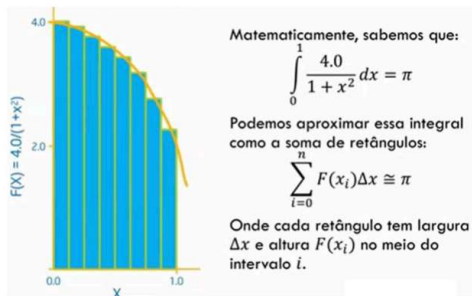
Apenas as afirmativas I e III estão corretas

Questão 7

Completo

Atingiu 1,0 de 1,0

O programa a seguir é uma implementação do cálculo da função Pi em modo serial, usando único espaço de endereçamento. Com base neste código crie uma versão CUDA, de modo que o valor de PI seja obtido por cálculos realizados na GPU, obedecendo a melhor relação possível entre blocos e threads por bloco.



```
1  #include <stdio.h>
2  #define NUM_STEPS 8000000
3
4  int main(void) {
5      double x, pi, sum=0.0;
6      double step;
7
8      step = 1.0/(double) NUM_STEPS;
9      for (int i=0; i<NUM_STEPS; i++) {
10         x = (i+0.5) * step;
11         sum+=4/(1.0+x*x);
12     } /*fim-for */
13     pi = sum*step;
14     printf("Pi = %f\n", pi);
15 } /*fim-main */
```

Obs.: o arquivo de resposta deve ser nomeado com a matrícula do aluno, por exemplo, 21000894.cu

 [190089601.cu](#)

Comentário: